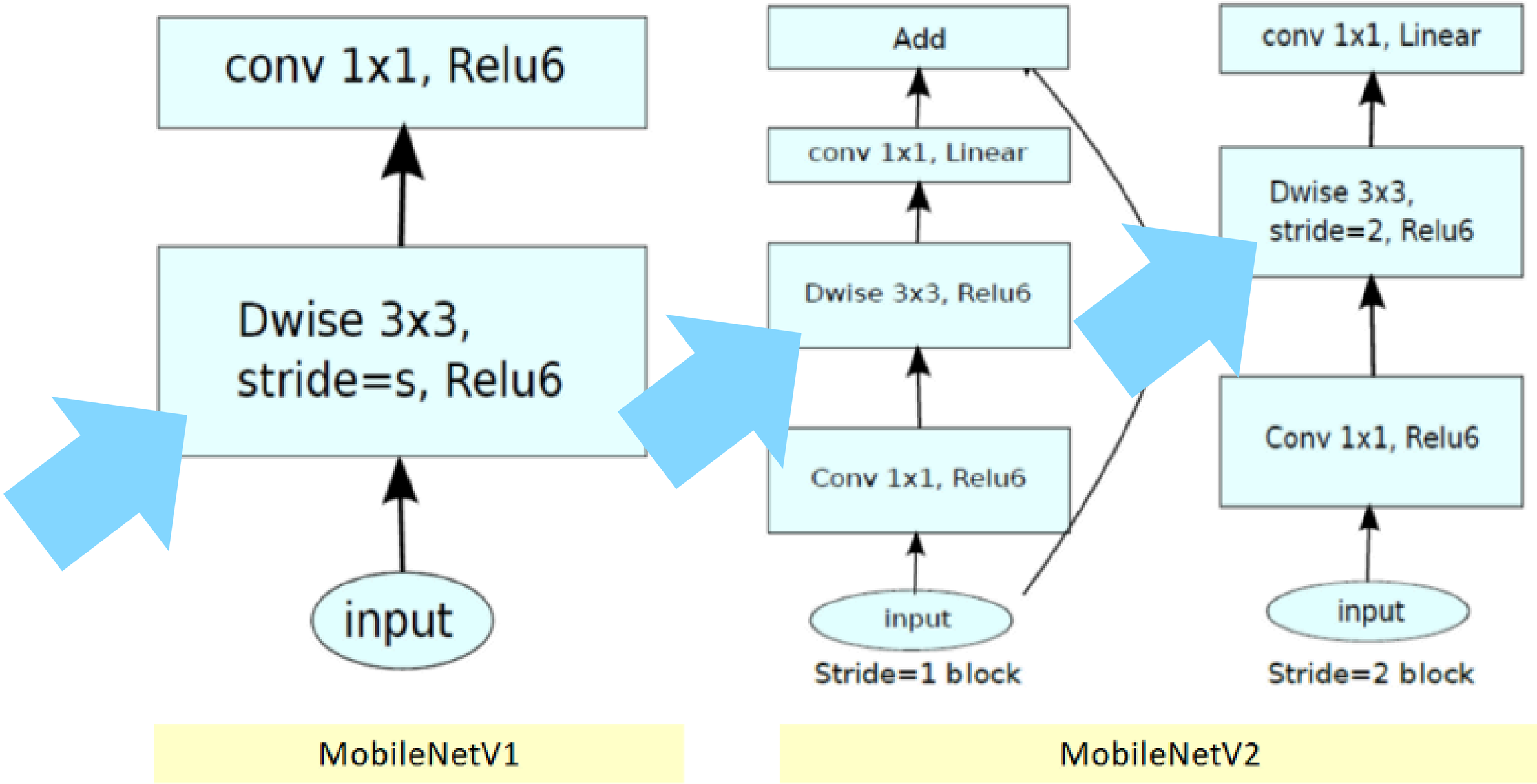
MobileNet Custom Kernel

custom kernel 1

- 3×3 depthwise konvolüsyonlar için **custom FP16 CUDA** kernel geliştirildi.
- Kernel yalnızca sabit NCHW veri düzenini ve depthwise hesaplamayı destekleyecek şekilde sadeleştirildi. (N batch boyutu, C kanal sayısı, H ve W ise resmin boyutları)
- **Her output** elemanı **tek bir CUDA thread** tarafından hesaplandı.
- Kanal bazlı filtreleme doğrudan uygulanarak **gereksiz kontrol ve dallanmalar kaldırıldı.**
- Hesaplamalar **FP32'de biriktirilip FP16 olarak yazılarak** sayısal doğruluk korundu.
- PyTorch'un genel amaçlı depthwise kernel'lerine kıyasla kernel seviyesinde belirgin performans kazanımı elde edildi.


conv 1x1, Relu6
Dwise 3x3,
stride=s, Relu6
input
Add
conv 1x1, Linear
Dwise 3x3, Relu6
Conv 1x1, Relu6
input
Stride=1 block
conv 1x1, Linear
Dwise 3x3,
stride=2, Relu6
Conv 1x1, Relu6
input
Stride=2 block
MobileNetV1
MobileNetV2

Performans

Latency

```
PyTorch FP16 depthwise conv: 0.018766 ms

Custom CUDA kernel: 0.009842 ms

Speedup (PyTorch / Custom): 1.91×
```

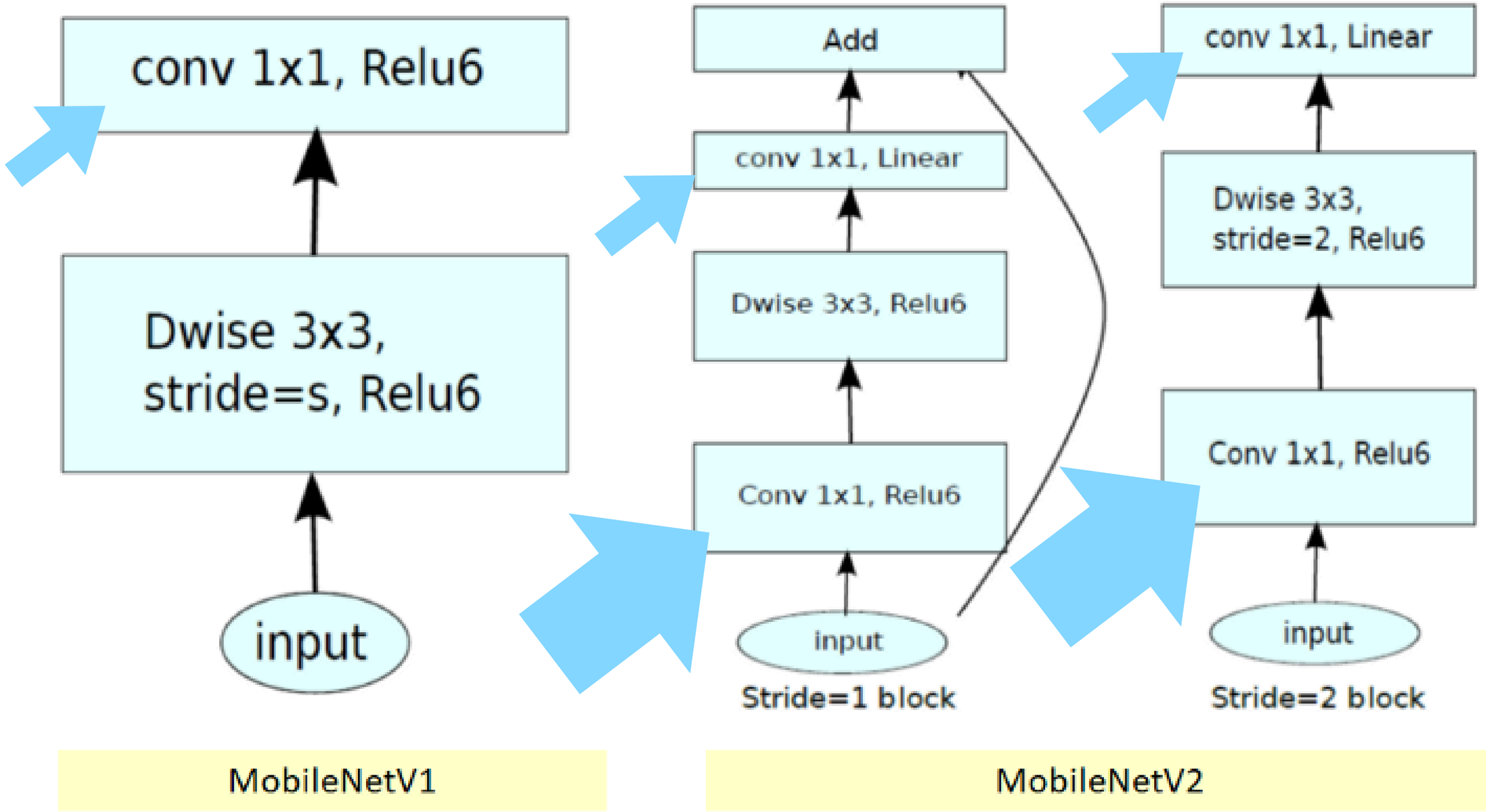
Throughput

```
C=16, H=W=112 | PT throughput=64379/s | CU throughput=104613/s | speedup=1.62×
C=32, H=W=56 | PT throughput=54448/s | CU throughput=102617/s | speedup=1.88×
C=64, H=W=28 | PT throughput=52405/s | CU throughput=104384/s | speedup=1.99×
```

```
C= 16, H=W=112 | PT=0.015255 ms | CU=0.009584 ms | speedup=1.59×
C= 32, H=W= 56 | PT=0.018718 ms | CU=0.009536 ms | speedup=1.96×
C= 64, H=W= 28 | PT=0.019701 ms | CU=0.009531 ms | speedup=2.07×
```

custom kernel 2

- 1×1 pointwise konvolüsyonlar için **FP16 custom CUDA** kernel geliştirildi.
- Kernel, **kanal boyutunda dot-product** hesaplamasına özel olarak tasarlandı.
- Sadece MobileNetV2’nin kullandığı **ağırlık ve veri yerleşimi** hedeflenerek genel amaçlı overhead azaltıldı.
- Her output elemanı için tüm input kanallarında **FP32 birikimli çarpma-toplama** yapıldı.
- Sonuçlar **FP16 formatına dönüştürülerek** doğruluk korunurken performans artırıldı.
- Compute-yoğun yapısı nedeniyle speedup sınırlı kalmasına rağmen tutarlı performans iyileştirmeleri sağlandı.


conv 1x1, Relu6
Dwise 3x3,
stride=s, Relu6
input
Add
conv 1x1, Linear
Dwise 3x3, Relu6
Conv 1x1, Relu6
input
Stride=1 block
conv 1x1, Linear
Dwise 3x3,
stride=2, Relu6
Conv 1x1, Relu6
input
Stride=2 block
MobileNetV1
MobileNetV2

Performans

Latency and Troughput

```
Pointwise PT latency: 0.0372 ms
Pointwise CU latency: 0.0235 ms
PT throughput: 26918 samples/s
CU throughput: 42516 samples/s
Speedup: 1.58×
```

```
C= 16, H=W=112 | PT=0.021380 ms | CU=0.010069 ms | PT tp=46772/s | CU tp=99315/s | speedup=2.12×
C= 32, H=W= 56 | PT=0.021131 ms | CU=0.009964 ms | PT tp=47323/s | CU tp=100366/s | speedup=2.12×
C= 64, H=W= 28 | PT=0.025985 ms | CU=0.009973 ms | PT tp=38484/s | CU tp=100273/s | speedup=2.61×
```

MobileNet ON DIFFERENT GPU'S

GOOGLE COLAB A100 GPU

```
1 !nvidia-smi
2
```

```
Mon Dec 15 19:48:28 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA A100-SXM4-80GB          Off |   00000000:00:05.0 Off |                    0 |
| N/A   32C    P0             56W /  400W |       0MiB /  81920MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

L4 GPU

```
1 !nvidia-smi
2
```

```
Mon Dec 15 20:27:34 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA L4                      Off |   00000000:00:03.0 Off |                    0 |
| N/A   45C    P8             12W /   72W |       0MiB /  23034MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

T4 GPU

```
1 !nvidia-smi
2
```

```
Mon Dec 15 20:46:46 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   34C    P8             11W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

A100

- MobileNetV2 için aşırı güçlü, çoğu kaynağı boşa gider.
- Büyük batch’lerde throughput çok yüksek olur.
- Asıl gücü büyük CNN ve Transformer modellerinde ortaya çıkar.
- MobileNetV2 gibi hafif modeller için maliyet/performans oranı kötü.

T4

- Uzun süredir inference için kullanılan klasik GPU.
- MobileNetV2 çalıştırır ama:
  - Latency L4’ten daha yüksek
  - Verimlilik daha düşük
- Güncel sistemlerde yerini L4’e bırakıyor.

L4

- MobileNetV2 inference için en dengeli GPU.
- Düşük latency, yüksek throughput ve düşük güç tüketimi.
- FP16 ve INT8 optimizasyonlarında çok başarılı.
- TensorRT ile birlikte kullanıldığında ideal production inference GPU’su.

```
=== FINAL RESULTS ===
Engine     Latency (ms)    Throughput (FPS)
FP32       0.3714          2692.36
FP16       0.2992          3342.45
```

A100

```
=== FINAL RESULTS ===
Engine     Latency (ms)    Throughput (FPS)
FP32       0.2857          3500.44
FP16       0.2577          3879.74
```

L4

```
=== FINAL RESULTS ===
Engine     Latency (ms)    Throughput (FPS)
FP32       0.6553          1526.03
FP16       0.3447          2900.84
```

T4

A100

```
===== FP32 Benchmarks =====
Batch 1:  Batch Lat = 0.361 ms,  Per-img = 0.3612 ms,  FPS = 2768.2
Batch 2:  Batch Lat = 0.362 ms,  Per-img = 0.1809 ms,  FPS = 5526.5
Batch 4:  Batch Lat = 0.362 ms,  Per-img = 0.0904 ms,  FPS = 11062.3
Batch 8:  Batch Lat = 0.361 ms,  Per-img = 0.0451 ms,  FPS = 22186.9
Batch 16:  Batch Lat = 0.361 ms,  Per-img = 0.0226 ms,  FPS = 44335.1
Batch 32:  Batch Lat = 0.359 ms,  Per-img = 0.0112 ms,  FPS = 89025.2
Batch 64:  Batch Lat = 0.361 ms,  Per-img = 0.0056 ms,  FPS = 177325.6

===== FP16 Benchmarks =====
Batch 1:  Batch Lat = 0.290 ms,  Per-img = 0.2895 ms,  FPS = 3454.0
Batch 2:  Batch Lat = 0.290 ms,  Per-img = 0.1449 ms,  FPS = 6901.4
Batch 4:  Batch Lat = 0.290 ms,  Per-img = 0.0724 ms,  FPS = 13811.3
Batch 8:  Batch Lat = 0.290 ms,  Per-img = 0.0362 ms,  FPS = 27594.7
Batch 16:  Batch Lat = 0.290 ms,  Per-img = 0.0181 ms,  FPS = 55185.6
Batch 32:  Batch Lat = 0.290 ms,  Per-img = 0.0091 ms,  FPS = 110332.0
Batch 64:  Batch Lat = 0.289 ms,  Per-img = 0.0045 ms,  FPS = 221275.1
```

T4

```
===== FP32 Benchmarks =====
Batch 1:  Batch Lat = 0.649 ms,  Per-img = 0.6485 ms,  FPS = 1542.0
Batch 2:  Batch Lat = 0.645 ms,  Per-img = 0.3227 ms,  FPS = 3099.2
Batch 4:  Batch Lat = 0.648 ms,  Per-img = 0.1620 ms,  FPS = 6173.2
Batch 8:  Batch Lat = 0.648 ms,  Per-img = 0.0810 ms,  FPS = 12351.7
Batch 16:  Batch Lat = 0.646 ms,  Per-img = 0.0404 ms,  FPS = 24761.8
Batch 32:  Batch Lat = 0.647 ms,  Per-img = 0.0202 ms,  FPS = 49442.4
Batch 64:  Batch Lat = 0.647 ms,  Per-img = 0.0101 ms,  FPS = 98941.5

===== FP16 Benchmarks =====
Batch 1:  Batch Lat = 0.333 ms,  Per-img = 0.3327 ms,  FPS = 3005.9
Batch 2:  Batch Lat = 0.331 ms,  Per-img = 0.1653 ms,  FPS = 6047.9
Batch 4:  Batch Lat = 0.329 ms,  Per-img = 0.0822 ms,  FPS = 12169.0
Batch 8:  Batch Lat = 0.331 ms,  Per-img = 0.0414 ms,  FPS = 24182.9
Batch 16:  Batch Lat = 0.331 ms,  Per-img = 0.0207 ms,  FPS = 48319.5
Batch 32:  Batch Lat = 0.328 ms,  Per-img = 0.0103 ms,  FPS = 97510.1
Batch 64:  Batch Lat = 0.330 ms,  Per-img = 0.0052 ms,  FPS = 193882.5
```

```
===== FP32 Benchmarks =====
Batch 1:  Batch Lat = 0.267 ms,  Per-img = 0.2674 ms,  FPS = 3739.8
Batch 2:  Batch Lat = 0.267 ms,  Per-img = 0.1337 ms,  FPS = 7477.3
Batch 4:  Batch Lat = 0.267 ms,  Per-img = 0.0667 ms,  FPS = 14991.4
Batch 8:  Batch Lat = 0.268 ms,  Per-img = 0.0334 ms,  FPS = 29901.7
Batch 16:  Batch Lat = 0.267 ms,  Per-img = 0.0167 ms,  FPS = 59834.4
Batch 32:  Batch Lat = 0.267 ms,  Per-img = 0.0084 ms,  FPS = 119639.3
Batch 64:  Batch Lat = 0.267 ms,  Per-img = 0.0042 ms,  FPS = 239279.3

===== FP16 Benchmarks =====
Batch 1:  Batch Lat = 0.239 ms,  Per-img = 0.2393 ms,  FPS = 4178.3
Batch 2:  Batch Lat = 0.239 ms,  Per-img = 0.1195 ms,  FPS = 8364.8
Batch 4:  Batch Lat = 0.240 ms,  Per-img = 0.0600 ms,  FPS = 16663.3
Batch 8:  Batch Lat = 0.238 ms,  Per-img = 0.0298 ms,  FPS = 33595.2
Batch 16:  Batch Lat = 0.238 ms,  Per-img = 0.0149 ms,  FPS = 67089.6
Batch 32:  Batch Lat = 0.239 ms,  Per-img = 0.0075 ms,  FPS = 133863.0
Batch 64:  Batch Lat = 0.241 ms,  Per-img = 0.0038 ms,  FPS = 265565.6
```

L4

```
PyTorch Latency: 22.01592445373535 ms
PyTorch FPS: 45.42166748897679
ONNX Runtime Latency: 2.5336303710937 5 ms
ONNX Runtime FPS: 394.6905639469056
TensorRT FP16 Latency: 0.3492503166198 7305 ms
TensorRT FP16 FPS: 2863.275858067176
```

A100

```
PyTorch Latency: 21.297357082366943 ms
PyTorch FPS: 46.95418291258053 5
ONNX Runtime Latency: 3.946469783782959 ms
ONNX Runtime FPS: 253.39101900874866
TensorRT FP16 Latency: 0.2383933067321 7773 ms
TensorRT FP16 FPS: 4194.748643356195
```

L4

```
PyTorch Latency: 24.031813144683838 ms
PyTorch FPS: 41.611508627313604
ONNX Runtime Latency: 3.993312835693 3594 ms
ONNX Runtime FPS: 250.41864766058828
TensorRT FP16 Latency: 0.6218981742858887 ms
TensorRT FP16 FPS: 1607.9802793251113
```

T4

```
PyTorch FP16 depthwise conv: 0.018766 ms
Custom CUDA kernel: 0.009842 ms
Speedup (PyTorch / Custom): 1.91×
```

A100

```
PyTorch FP16 depthwise conv: 0.018167 ms
Custom CUDA kernel: 0.009471 ms
Speedup (PyTorch / Custom): 1.92×
```

L4

```
PyTorch FP16 depthwise conv: 0.020027 ms
Custom CUDA kernel: 0.031301 ms
Speedup (PyTorch / Custom): 0.64×
```

T4

```
C= 16, H=W=112 | PT=0.015255 ms | CU=0.009584 ms | speedup=1.59×
C= 32, H=W= 56 | PT=0.018718 ms | CU=0.009536 ms | speedup=1.96×
C= 64, H=W= 28 | PT=0.019701 ms | CU=0.009531 ms | speedup=2.07×
```

A100

```
C= 16, H=W=112 | PT=0.015041 ms | CU=0.009133 ms | speedup=1.65×
C= 32, H=W= 56 | PT=0.018223 ms | CU=0.009202 ms | speedup=1.98×
C= 64, H=W= 28 | PT=0.019151 ms | CU=0.009190 ms | speedup=2.08×
```

L4

```
C= 16, H=W=112 | PT=0.028569 ms | CU=0.018755 ms | speedup=1.52×
C= 32, H=W= 56 | PT=0.017115 ms | CU=0.008464 ms | speedup=2.02×
C= 64, H=W= 28 | PT=0.017158 ms | CU=0.008468 ms | speedup=2.03×
```

T4

```
Pointwise PT latency: 0.0372 ms
Pointwise CU latency: 0.0235 ms
PT throughput: 26918 samples/s
CU throughput: 42516 samples/s
Speedup: 1.58×
```

A100

```
Pointwise PT latency: 0.0227 ms
Pointwise CU latency: 0.0138 ms
PT throughput: 44051 samples/s
CU throughput: 72692 samples/s
Speedup: 1.65×
```

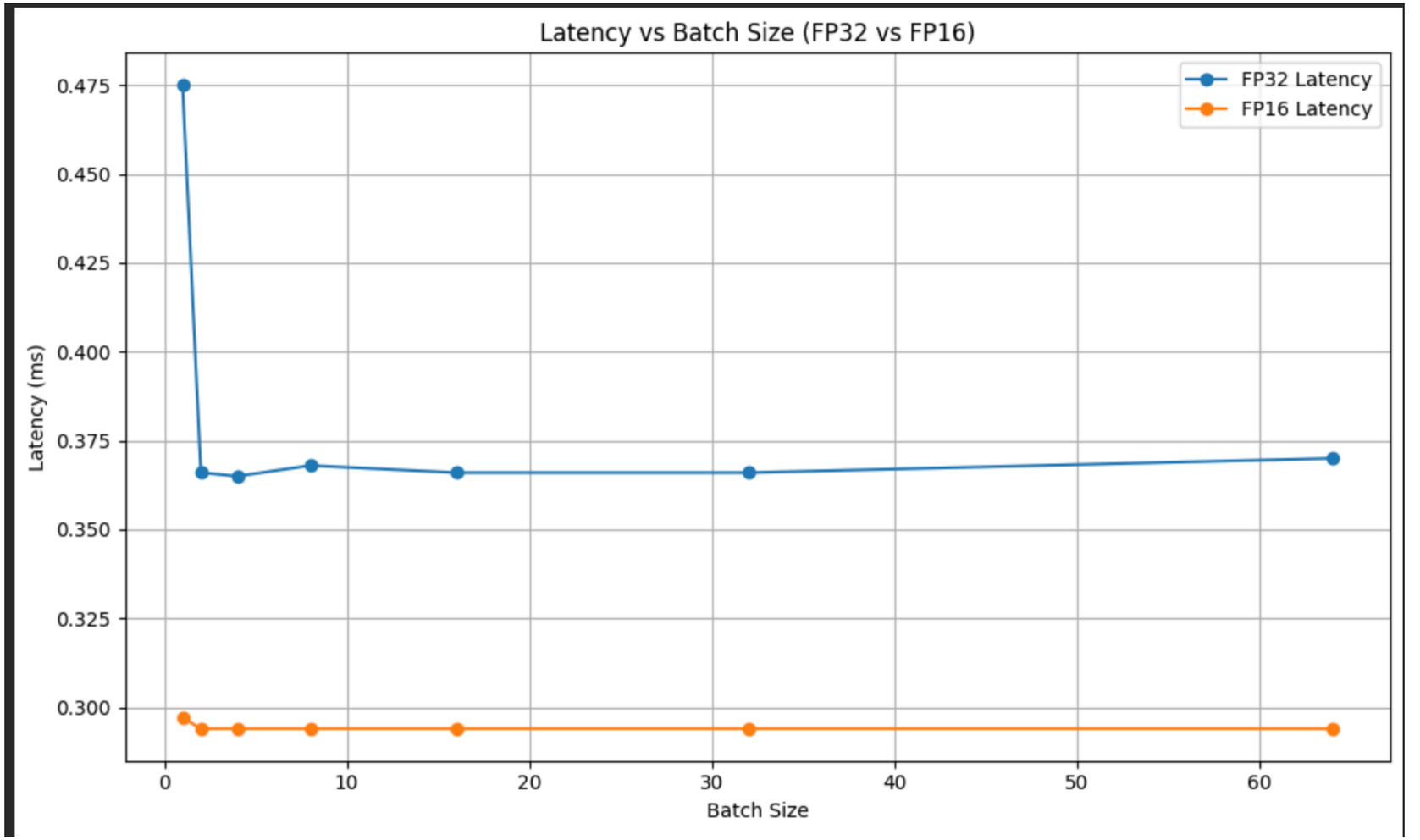
L4

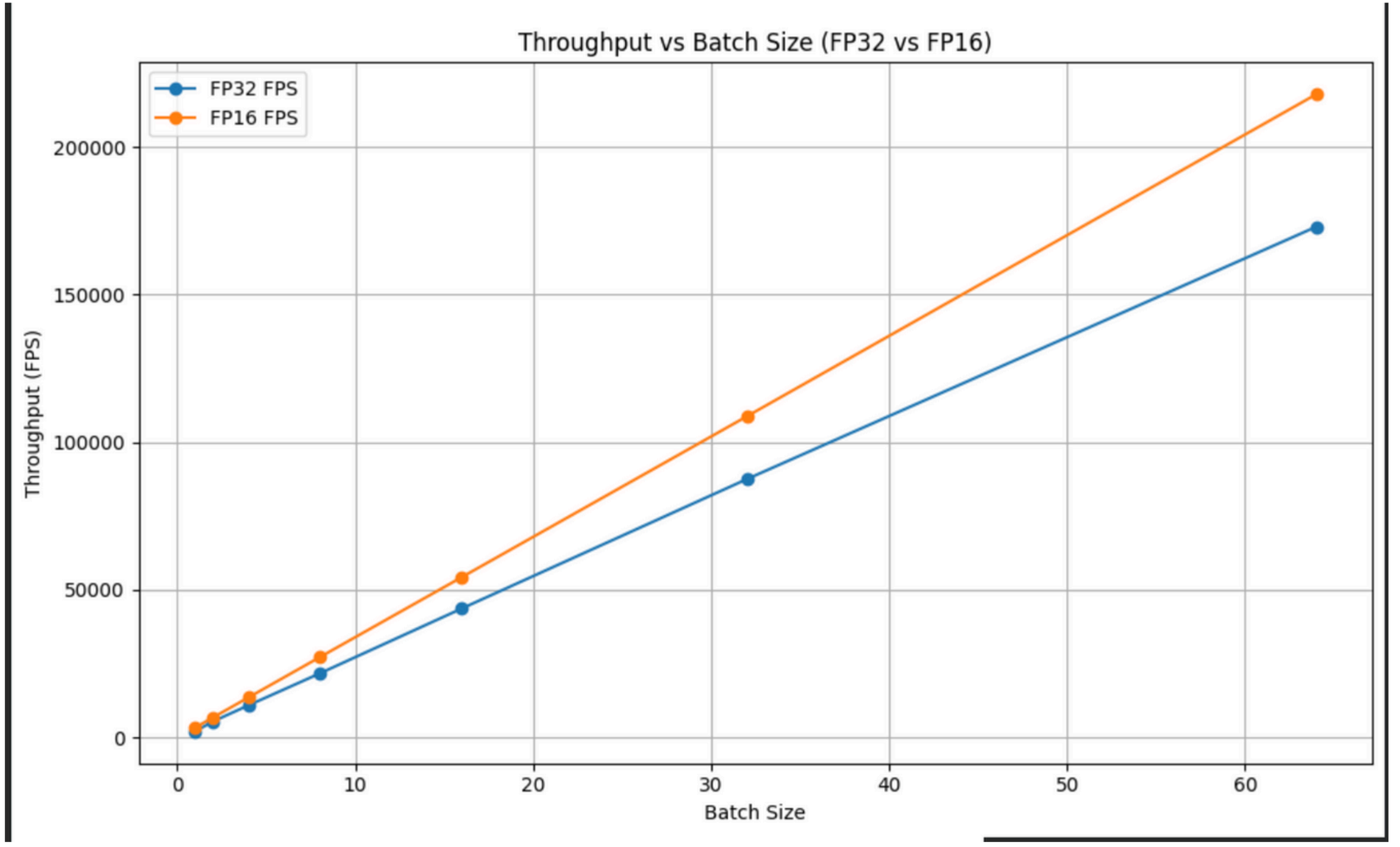
```
Pointwise PT latency: 0.0204 ms
Pointwise CU latency: 0.0485 ms
PT throughput: 48927 samples/s
CU throughput: 20613 samples/s
Speedup: 0.42×
```

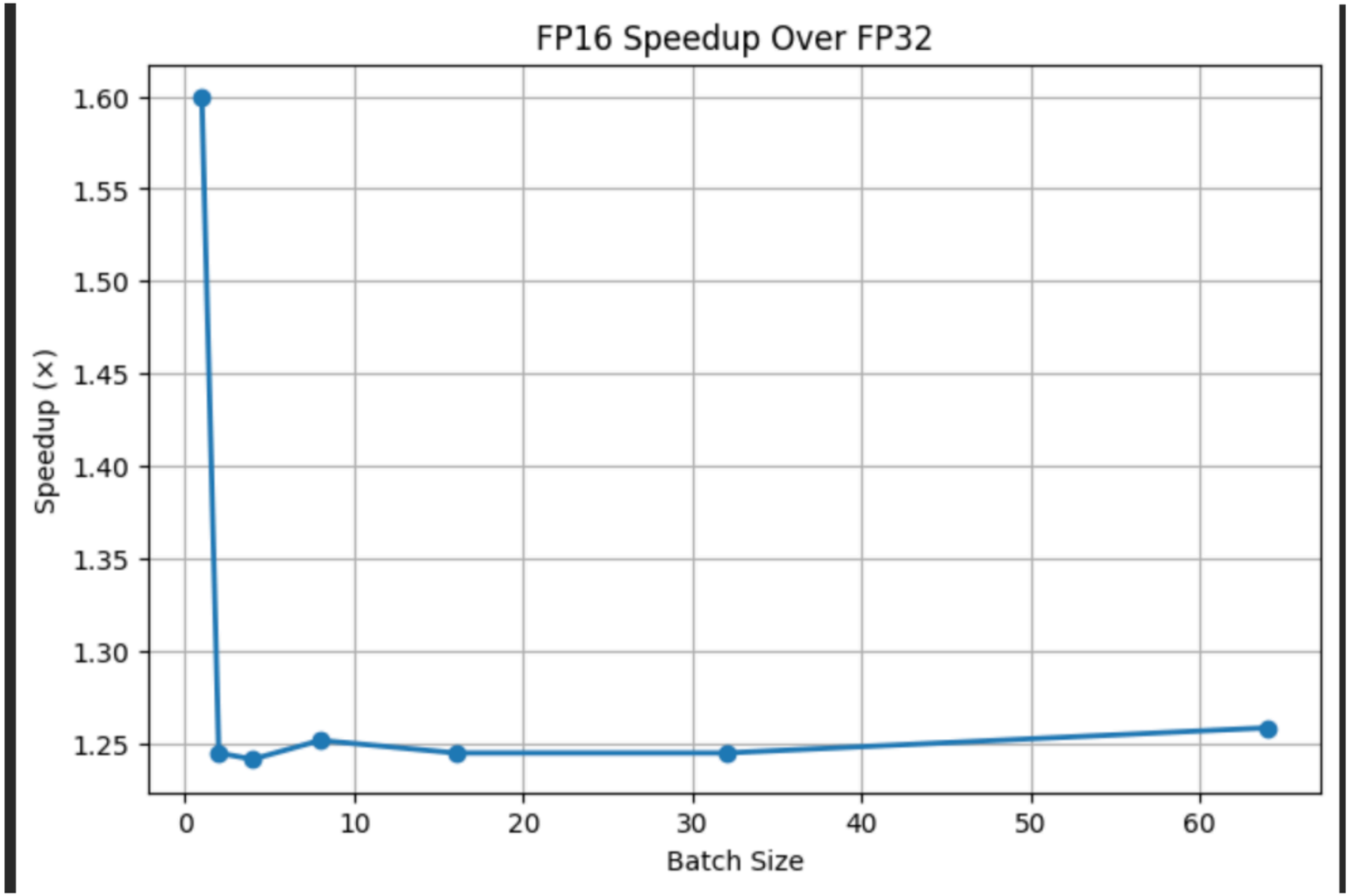
T4

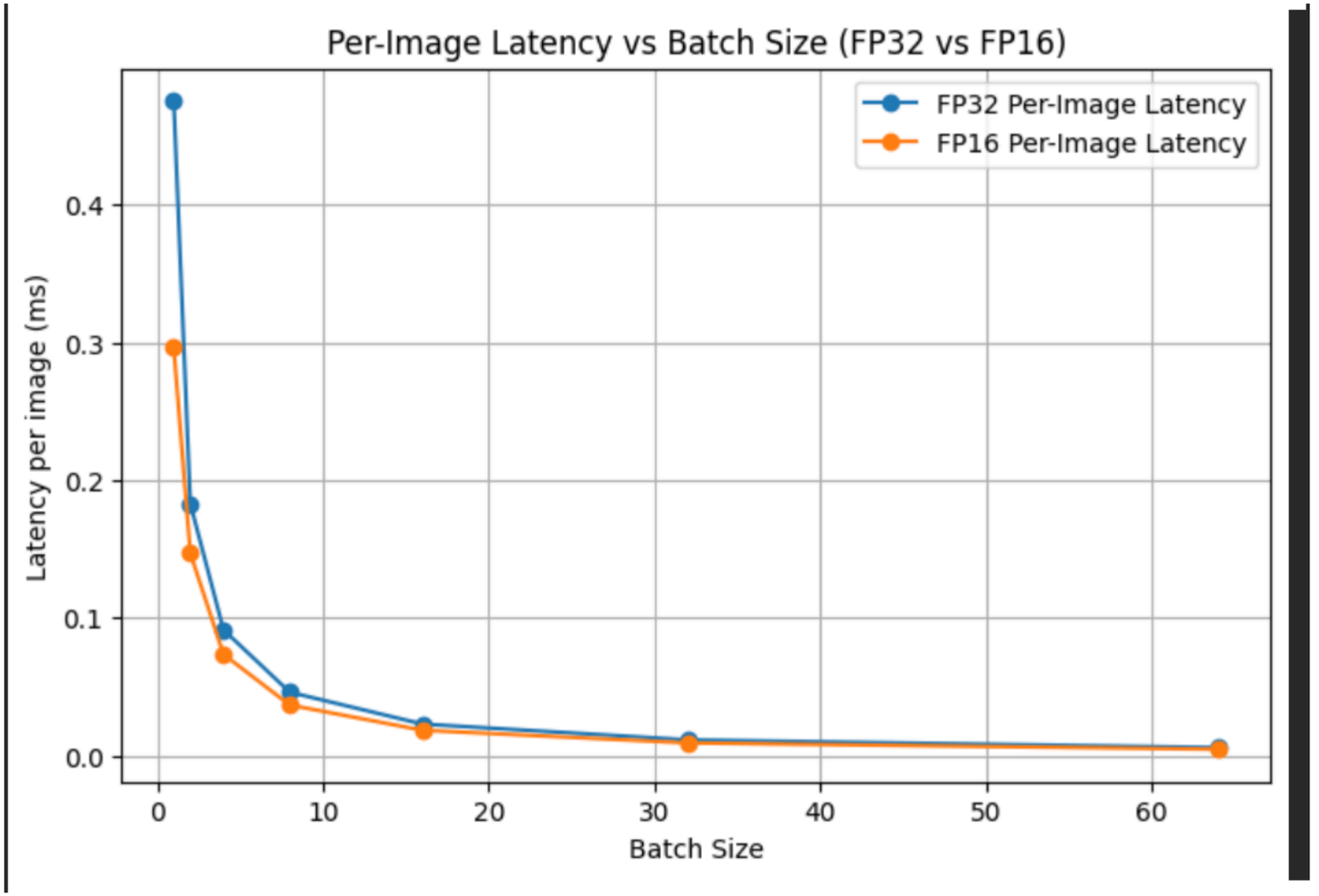
Graphs on A100

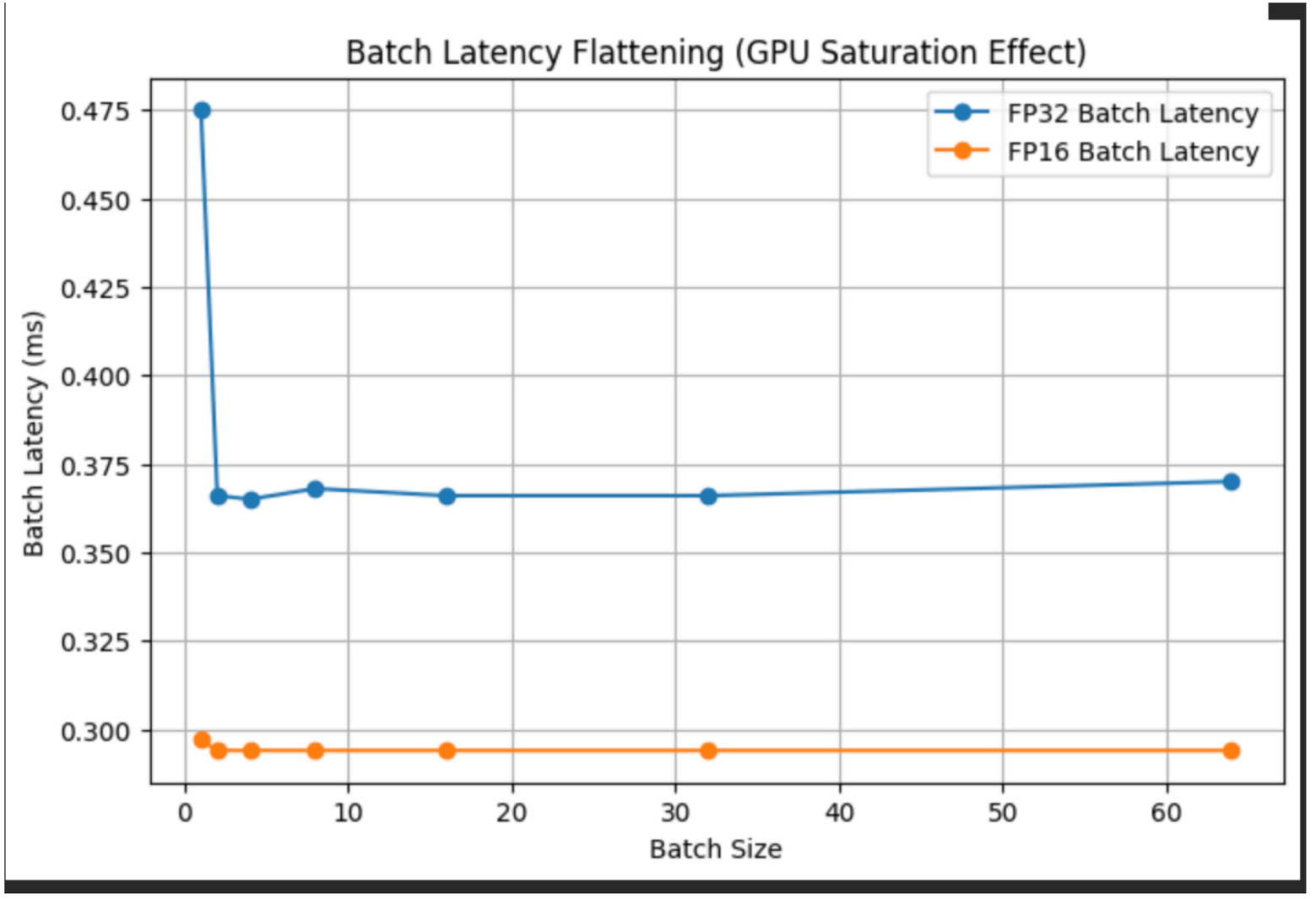
FP16/32
PyTorch - ONNX - TensorRT
BATCH SIZES

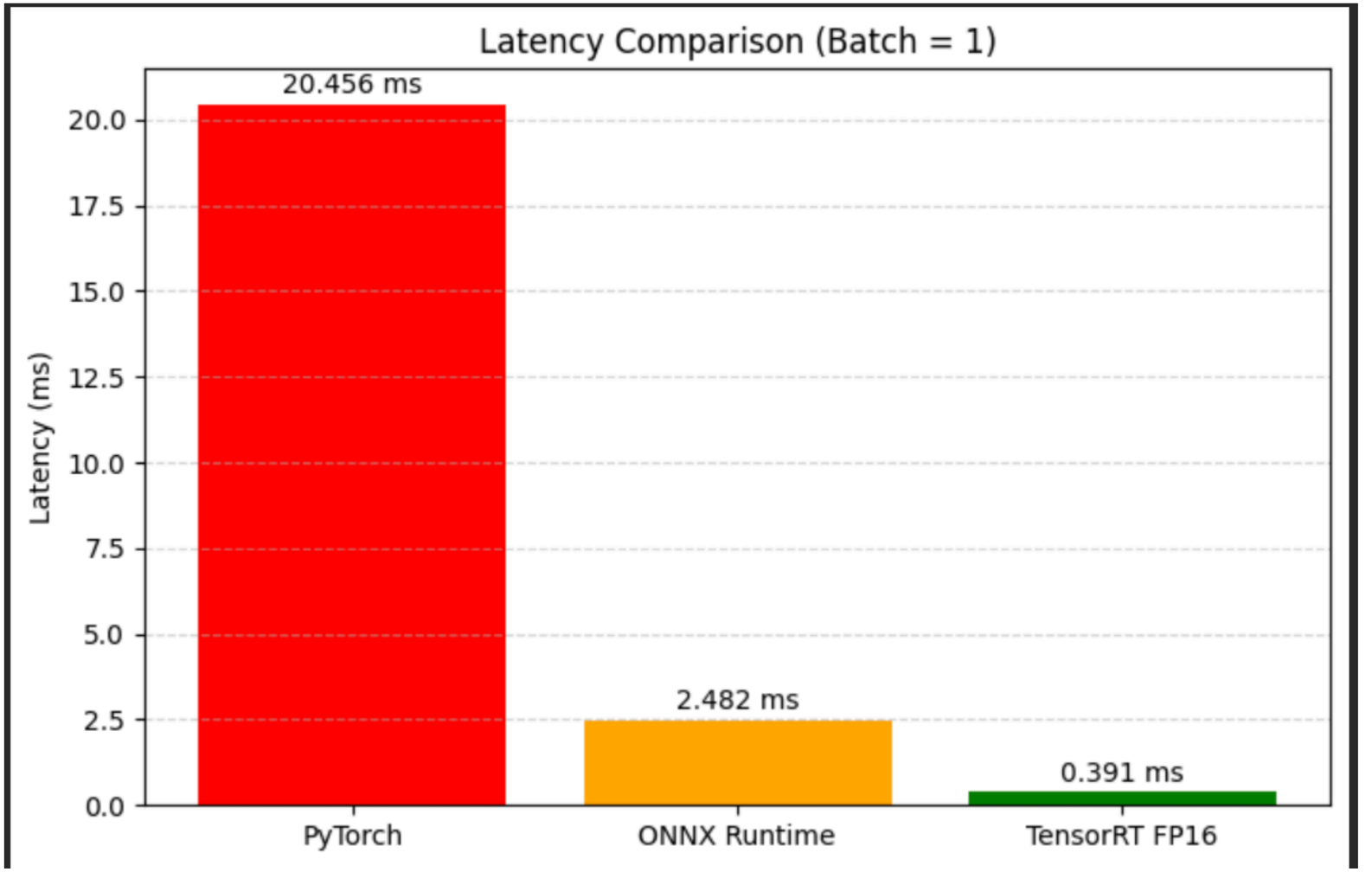

Latency vs Batch Size (FP32 vs FP16)
FP32 Latency
FP16 Latency
Latency (ms)
0.475
0.450
0.425
0.400
0.375
0.350
0.325
0.300
0
10
20
30
40
50
60
Batch Size

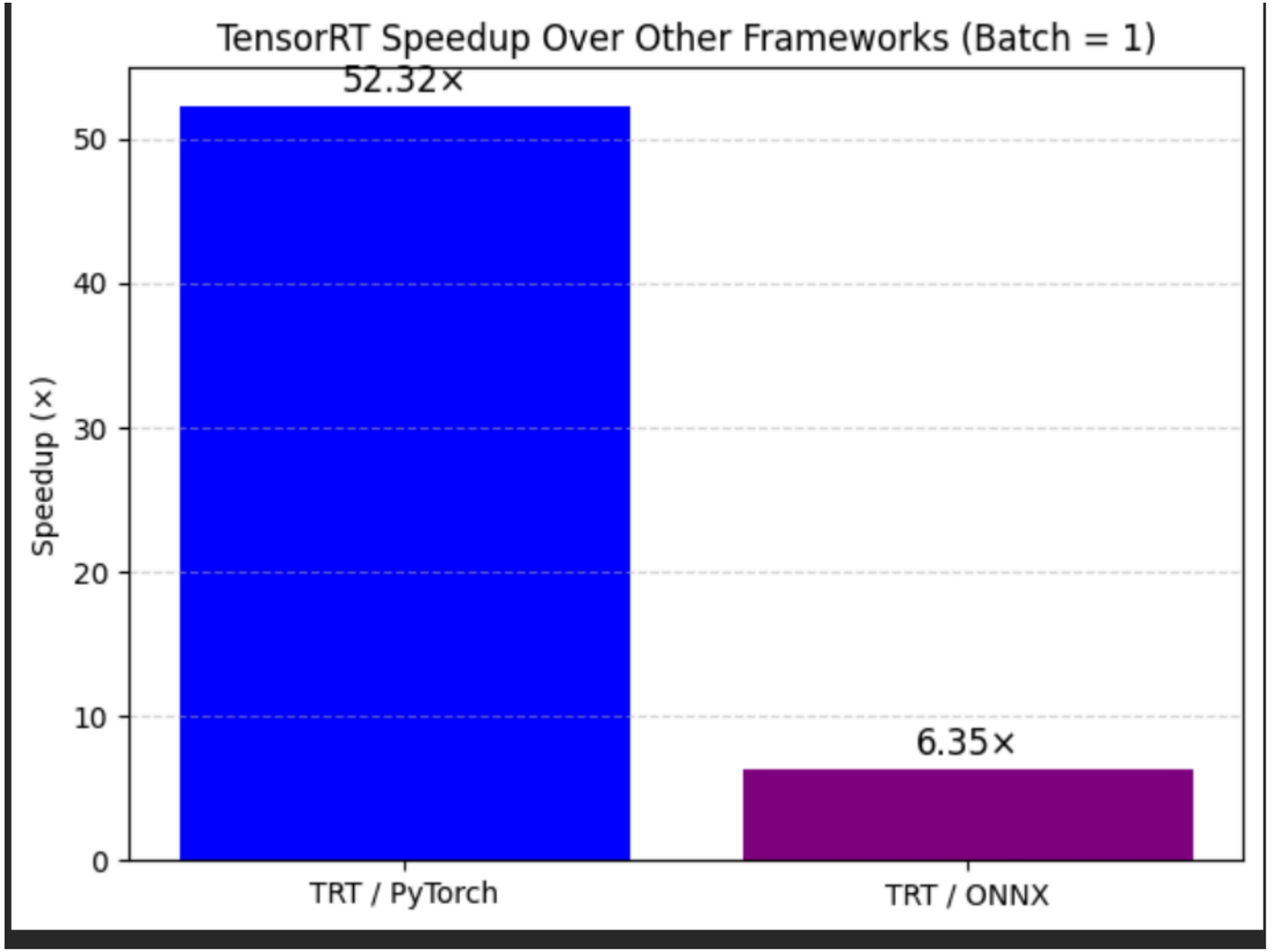

Throughput vs Batch Size (FP32 vs FP16)
FP32 FPS
FP16 FPS
Throughput (FPS)
200000
150000
100000
50000
0
0
10
20
30
40
50
60
Batch Size

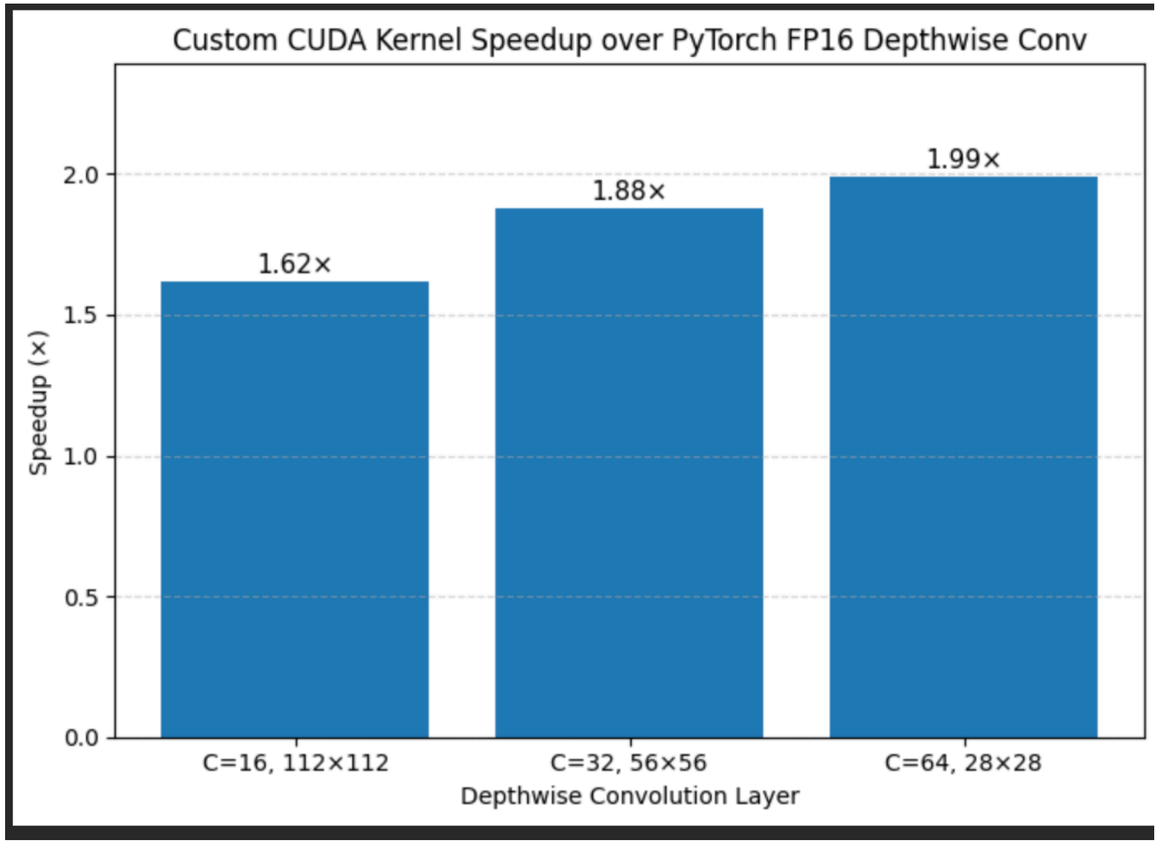

FP16 Speedup Over FP32
Speedup (×)
1.60
1.55
1.50
1.45
1.40
1.35
1.30
1.25
0
10
20
30
40
50
60
Batch Size

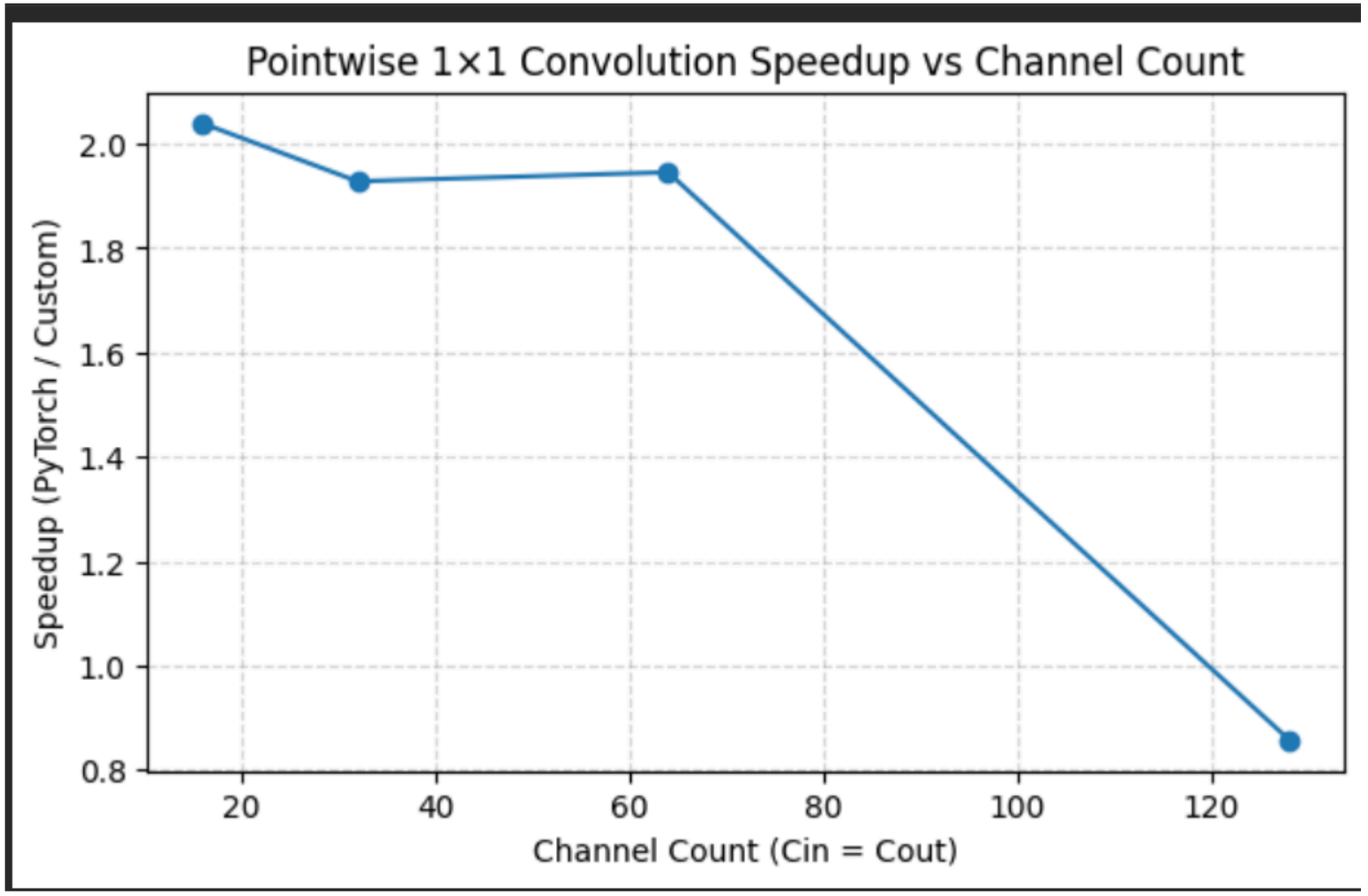

Per-Image Latency vs Batch Size (FP32 vs FP16)
FP32 Per-Image Latency
FP16 Per-Image Latency
Latency per image (ms)
0.0
0.1
0.2
0.3
0.4
0
10
20
30
40
50
60
Batch Size


Batch Latency Flattening (GPU Saturation Effect)
FP32 Batch Latency
FP16 Batch Latency
Batch Latency (ms)
0.475
0.450
0.425
0.400
0.375
0.350
0.325
0.300
0
10
20
30
40
50
60
Batch Size


Latency Comparison (Batch = 1)
Latency (ms)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
20.456 ms
2.482 ms
0.391 ms
PyTorch
ONNX Runtime
TensorRT FP16


TensorRT Speedup Over Other Frameworks (Batch = 1)
52.32×
6.35×
Speedup (x)
0
10
20
30
40
50
TRT / PyTorch
TRT / ONNX

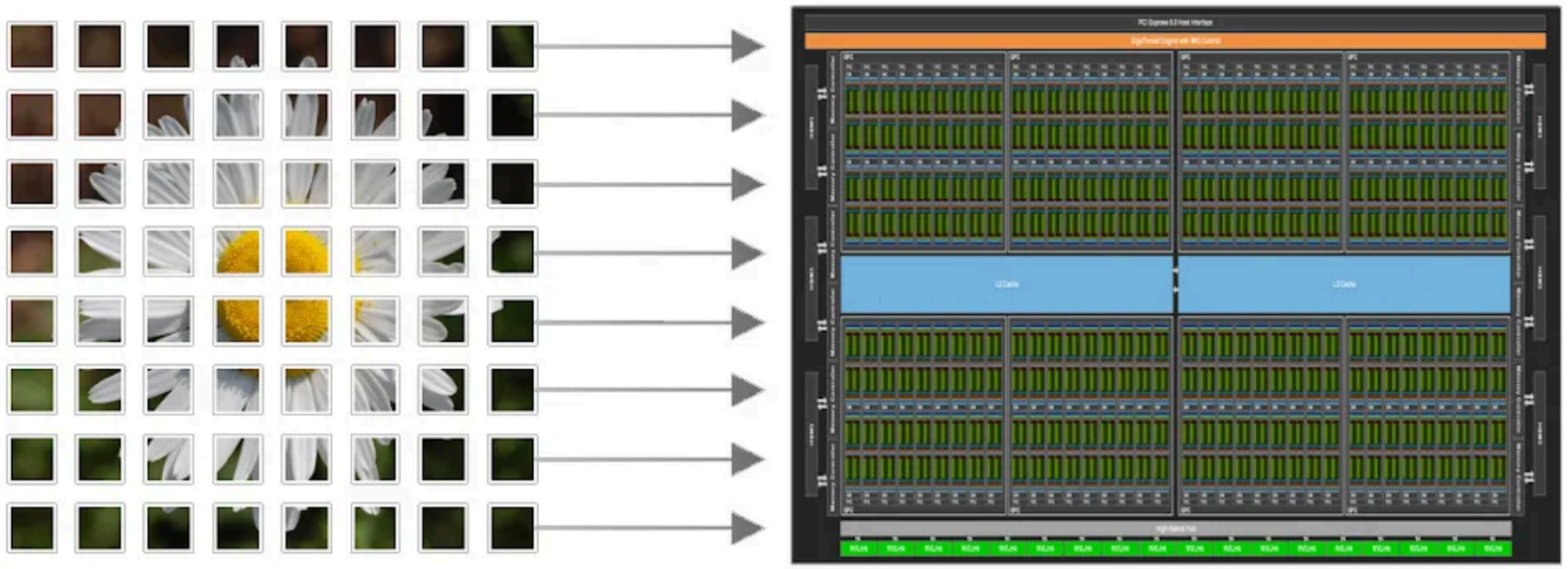

Custom CUDA Kernel Speedup over PyTorch FP16 Depthwise Conv
Speedup (×)
2.0
1.5
1.0
0.5
0.0
1.62×
1.88×
1.99×
C=16, 112×112
C=32, 56×56
C=64, 28×28
Depthwise Convolution Layer


Pointwise 1×1 Convolution Speedup vs Channel Count
Speedup (PyTorch / Custom)
2.0
1.8
1.6
1.4
1.2
1.0
0.8
20
40
60
80
100
120
Channel Count (Cin = Cout)

CUDA NEW VERSION

CUDA Tile Nedir ve Neyi Değiştiriyor

CUDA Tile, yapay zeka ve yüksek performanslı hesaplama iş yüklerinde kullanılan işlem akışını **thread seviyesinden veri bloğu seviyesine** taşıyan karo temelli bir modeldir.



Bu modelde geliştirici:

- Veriyi küçük parçalara, yani tile’lara böler
- Tile üzerinde yapılacak matematiksel işlemi tanımlar
- İşin GPU üzerinde nasıl dağıtılacağını CUDA runtime ve derleyiciye bırakır

Böylece her thread’in yürütme yolunu tek tek tanımlamaya gerek kalmaz.

SONRAKİ HAFTANIN HEDEFLERİ

- IOT CİHAZLARDA LATENCY VE THROUGHPUT ARAŞTIR
- HANGİ GPU, BATCH SIZE, MODEL (FP16,32) KULLANILABİLİR ONLARI BELİRLE
- PYTORCH, TENSORRT VE CUSTOM KERNEL’LARDAN HANGİLERİ KULLANILABİLİR ONLARI BELİRLE